SHREE VENKATESHWARA
HI-TECH ENGINEERING
COLLEGE (AUTONOMOUS)
GOBI-638 455

NAME OF THE LAB_____

University Register No_____

Name _____ Roll No_____

Branch_____ Batch_____

Certified that this is bonafide record of work done by the above student during the year 20 -20

LAB IN-CHARGE **HEAD OF THE DEPARTMENT**

Submitted for the practical Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

CONTENTS

Exp. No.	Date	Nameofthe Experiment	PageNo.	Marks	Faculty Signature
1		**Using commands like tcpdump, netstat, ifconfig,nslookupandtraceroute.Capturepin g and traceroute PDUs usinga network protocol analyzer and examine**			
2		**HTTP web client program to download a web page using TCP sockets**			
3		3a) **Applications using TCP sockets- Echo client and echo server**			
		3b) **Applications using TCP sockets- Chat**			
4		**Simulation of DNS using UDP Sockets**			
5		**Using a tool like Wireshark to capture packets and examine the packets**			
6		6a) **Address Resolutuion Protocol (ARP) using UDP**			
		6b) **Reverse Address Resolution Protocol(RARP) using UDP**			
7		**Network simulator (NS) and Simulation of Congestion Control Algorithms using NS**			
8		**TCP/UDP performance using Simulation tool.**			
9		9a) **Simulation of Distance Vector Routing Algorithm**			
		9b) **Simulation Of Link State Routing Algorithm**			
10		**Simulation of Error Detection Code (like CRC)**			

Ex.No:1	Using commands like tcpdump, netstat, ifconfig, nslookup and traceroute.Captureping and traceroute PDUs usinga network protocol analyzer and examine.
Date:	

Aim:

To use commands like tcpdump, netstat,ifconfig,nslookup and traceroute ping.

COMMANDS:

C:\>arp –a: ARP is short form of address resolution protocol, It will show the IP address of your computer along with the IP address and MAC address of your router.

C:\>hostname: This is the simplest of all TCP/IP commands.It simply displays the name of your computer.

C:\>ipconfig: The ip config command displays information about the host(the computer your sitting at)computer TCP/IP configuration.

C:\>ipconfig /all: This command displays detailed configuration information about your TCP/IP connection including Router,Gateway,DNS,DHCP,and type of Ethernet adapter in your system.

C:\>Ipconfig/renew:Using this command will renewall your IP addresses that you are currently (leasing) borrowing from the DHCP server. This command is a quick Problem solver if you are having connection issues,but does not work if you have been configured with a static IP address.

C:\>Ipconifg/release:This command allows you to drop the IP lease from the DHCP server.

C:\>ipconfig /flushdns: This command is only needed if you're having trouble with your networks DNS configuration. The best time to use this command is after network configuration frustration sets in,and you really need the computer to reply with flushed.

C:\>nbtstat–a: This command helps solve problems with Net BIOS name resolution.(Nbt stands for NetBIOS over TCP/IP)

C:\>netdiag:Netdiag is a network testing utility that performs a variety of network Diagnostic tests,allowing you to pinpoint problems in your network.Netdiagisn't installed by default, but can be installed from the Windows XP CD after saying no to the install. Navigate to the CDROM drive letter and open the support\tools folder on the XPCD and click the setup.exe icon in the support\tools folder.

C:\>netstat:Netstat displays a variety of statistics about a computers active TCP/IP connections. This tool is most useful when you're having trouble with TCP/IP applications such as HTTP, and FTP.

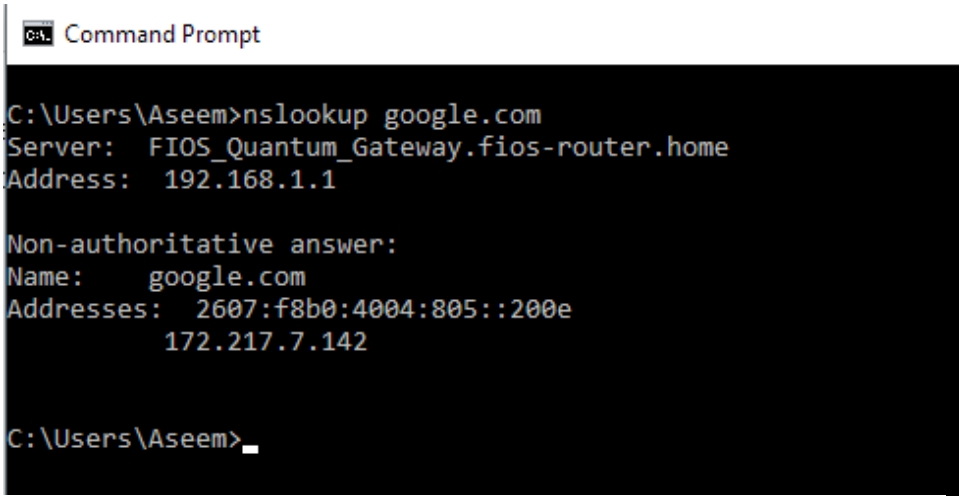
C:\>nslookup:Ns lookup isused for diagnosing DNS problems. If you can access are source by specifying an IP address but not it's DNS you have a DNS problem.

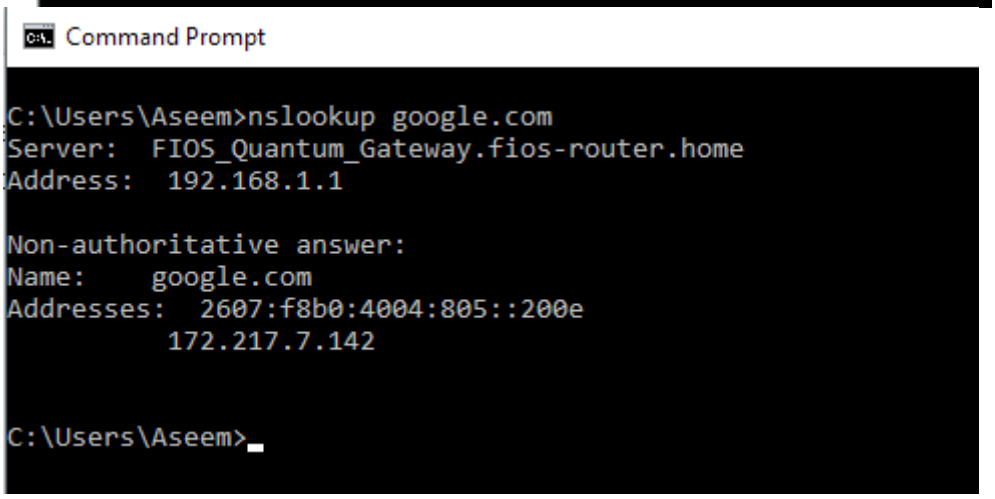
C:\>pathping: Path ping is unique to Window's, and is basically a combination of the Ping and Tracert commands. Pathping traces the route to the destination address then launches a 25second test of each router along the way,gathering statistics on the rate of

data loss along each hop.
C:\>ping: Ping is the most basic TCP/IP command, and it's the same as placing a phone call to your bestfriend.You pickup your telephone and dial a number,expecting your best friend to reply with ―Hello‖ on the other end. Computers make phone calls to each other over a network by using a Ping command. The Ping commands main purpose is to place A phone call to another computer on the network, and request an answer.Ping has 2 options it can use to place a phone call to another computer on the network. It can use the computers name or IP address.

C:\>route: The route command displays the computers routing table. A typical computer, with a single network interface, connected to a LAN, with a router is fairly simple and generally doesn't pose any network problems. But if you're having trouble accessing other computers on your network, you can use the route command to make sure the entries in the routing table are correct.

C:\>tracert:The tracert command displays a list of all the routers that a packet has to go through to get from the computer where tracert is run to any other computer on the internet.





RESULT:

Thus the above list of commands has been Executed.

Ex.No:2	HTTP web client program to download a web page using TCP sockets.
Date:	

AIM:

To write a HTTP web client program to download a web page using TCP sockets.

ALGORITHM:

1. Create a URL object and pass url as string to download the webpage.URL example = new URL(pass url of webpage you want to download)
2. Create Buffered Reader object and pass openStream(). Method of URL in Input Streamobject.
3. Create a string object to read each line one by one from stream.
4. Write each line in html file where web page will be downloaded.
5. Close all objects.
6. Catch exceptions if url failed to download.

```
import java.io.*;
import java.net.URL;
import java.net.Malformed URL Exception;

public class download{

   public static void Download Web Page(String webpage)
   {
      try{

         //Create URL object
         URL url =new URL(webpage);
         Buffered Reader readr =
           New Buffered Reader(new Input Stream Reader(url.open Stream()));

         // Enter filename in which you want to download
         Buffered Writer writer =
           New Buffered Writer(new File Writer("Download.html"));

         // read each line from stream till end
         String line;
         while ((line = readr.readLine()) != null) {
            writer.write(line);
         }

         readr.close();
         writer.close();
         System.out.println("Successfully Downloaded.");
```

```
        }

//Exceptions
        catch (Mal formed URL Exception mue)
         {
        System.out.println("Malformed URL Exception raised");
        }
        catch (IOException ie)
        {
        System.out.println("IO Exception raised");
        }
    }
    Public static voidmain(String args[])
        Throws IOException
    {
        String url = "https://www.geeksforgeeks.org/";
        Download Web Page(url);
    }
}
```

OUTPUT:

```
Successfully Downloaded.
```

RESULT:

Thus HTTP web client program to download a webpage using TCP sockets was developed and executed successfully.

Ex.No:3A	Applications using TCP sockets- Echo client and echo server
Date:	

AIM

To write a socket program for implementation of echo.

ALGORITHM

Client

1. Start
2. Create the TCP socket
3. Establish connection with the server
4. Get them Message to Reached from the user
5. Send the message to the server
6. Receive the message reached by the server
7. Display the message received from the server
8. Terminate the connection
9. Stop

Server

1. Start
2. Create TCP socket, make it a listening socket
3. Accept the connection request sent by the client for connection establishment
4. Receive the message sent by the client
5. Display the received message
6. Send the received message to the client from which it receives

Close the connection when client initiates termination and server becomes a listening server, Waiting for clients.

7. Stop.

PROGRAM:

EchoServer.java

```
import java.net.*;
import java.io.*;
public class EServer
{
Public static void main(String  args[])
{
Server Sockets=null;
String line;
Data Input Stream is;
Print Stream ps;
Socket c=null;
try
{
s=new Server Socket(9000);
}
catch(IO Exceptione)
{

System.out.println(e);
```

```
}
try
{
c=s.accept();
is=new Data InputStream(c.get InputStream());
ps=new Print Stream(c.get Output Stream());
while(true)
{
line=is.read Line();
ps.println(line);
}
}
catch(IO Exceptione)
{
System.out.println(e);
}
}
}
```

EClient.java

```
import java.net.*;
import java.io.*;
public classEClient
{
Public Static voidmain(String arg[])
{
Socketc=null;
String line;
Data InputStreamis,is1;
PrintStream os;
try
{
Inet Address ia=Inet Address.get LocalHost();
c=new Socket(ia,9000);
}
catch(IOExceptione)
{
System.out.println(e);
}
try
{
os=new Print Stream(c.get OutputStream());
is=new Data Input Stream(System.in);
is1=new Data Input
Stream(c.getInputStream());
while(true)
{
System.out.println("Client:");
line=is.read Line();
os.println(line);
System.out.println("Server:"+is1.readLine());
```

```
    }
  }
    catch(IOExceptione)
    {
    System.out.println("SocketClosed!");
    }
    }}
```

OUTPUT

Server

```
C:\ProgramFiles\Java\jdk1.5.0\bin>javacEServer.java
C:\Program Files\Java\jdk1.5.0\bin>java EServer
C:\Program Files\Java\jdk1.5.0\bin>
```

Client

```
C:\ProgramFiles\Java\jdk1.5.0\bin>javacEClient.java
C:\Program Files\Java\jdk1.5.0\bin>java EClient
Client: Hai Server
Server:HaiServer
Client: Hello
Server:Hello
Client:end
Server:end
Client:ds
Socket Closed
```

RESULT:

Thus the java application program for Echo client and Echo server using TCP Sockets was developed and executed successfully.

Ex.No:3B	Applications using TCP sockets- Chat
Date:	

AIM

To write a socket program for implementation of Chat program.

ALGORITHM

Client

1. Start
2. Create the UDP datagram socket
3. Get the request message to be sent from the user
4. Send the request message to the server
5. If the request message is―END‖go to step10
6. Wait for the reply message from the server
7. Receive the reply message sent by the server
8. Display the reply message received from the server
9. Repeat the steps from3 to 8
10. Stop

Server

1. Start
2. Create UDP datagram socket, make it a listening socket
3. Receive the request message sent by the client
4. If the received message is―END‖go to step10
5. Retrieve the client's IPaddress from the request message received
6. Display the received message
7. Get the reply message from the user
8. Send the reply message to the client
9. Repeat the steps from3 to 8.
10. Stop.

PROGRAM

UDPserver.java

```
importjava.io.*;
importjava.net.*;
class UDPserver
{
Public static Datagram Socketds;
public static byte buffer[]=new byte[1024];
public static int client port=789,serverport=790;
public static void main(String args[])throws Exception
{
ds=new Datagram Socket(client port);
System.out.println("press ctrl+c to quit the program");
Buffered Reader dis=new Buffered Reader(newInputStreamReader(System.in));
Inet Address ia=Inet Address.gey LocalHost();
while(true)
{
```

```
Data gramPacket p=new Datagram Packet(buffer,buffer.length);
ds.receive(p);
    String psx=new String(p.getData(),0,p.getLength());
    System.out.println("Client:" + psx);
    System.out.println("Server:");
    Stringstr=dis.readLine();
    if(str.equals("end"))
    break;
    buffer=str.get Bytes();
    ds.send(new Datagram Packet(buffer,str.length(),ia,server port));
    }
    }
    }
```

UDPclient.java

```
    import java .io.*;
    importjava.net.*;
    class UDPclient
    {
    Public static Datagram Socketds;
    Public static int client port=789,serverport=790;
    Public static void main(String args[])throws Exception
    {
    byte buffer[]=new byte[1024];
    ds=new Datagram Socket(serverport);
    Buffered Readerdis=new Buffered Reader(new Input Stream Reader(System.in));
    System.out.println("server waiting");
    Inet Address ia=Inet Address.get LocalHost();
    while(true)
    {
    System.out.println("Client:");
    String str=dis.readLine();
    if(str.equals("end"))
    break;
    buffer=str.getBytes();
    ds.send(new DatagramPacket(buffer,str.length(),ia,clientport));
    Datagram Packetp=newDatagram Packet(buffer,buffer.length);
    ds.receive(p);
    String psx=newString(p.getData(),0,p.getLength());
    System.out.println("Server:" + psx);
    }
    }
}
```

OUTPUT:

Server

```
C:\ProgramFiles\Java\jdk1.5.0\bin>javac UDPserver.java
C:\Program Files\Java\jdk1.5.0\bin>java UDPserver
pressctrl+c to quit the program
Client:Hai Server
Server:Hello Client
Client:How are You
Server:I am Fine
```

Client

```
C:\ProgramFiles\Java\jdk1.5.0\bin>javac UDPclient.java
C:\Program Files\Java\jdk1.5.0\bin>java UDPclient
server waiting
Client:Hai Server
Server:Hello Client
Client:How are You
Server:I am Fine
Client:end
```

RESULT:

Thus the java application program for chat using TCP Sockets was developed and executed successfully.

Ex.No:4	Simulation of DNS using UDP Sockets
Date:	

AIM:
To write a java program to Simulations of DNS using UDP Sockets

ALGORITHM
Server
1. Start
2. Create UDP datagram socket
3. Create a table that maps host name and IPaddress
4. Receive the host name from the client
5. Retrieve the client's IP address from the received datagram
6. Get the IP address mapped for the host name from the table.
7. Display the host name and corresponding IP address
8. Send the IP address for the requested host name to the client
9. Stop.

Client
1. Start
2. Create UDP datagram socket.
3. Get the host name from the client
4. Send the host name to the server
5. Wait for the reply from the server
6. Receive the reply datagram and read the IP address for the requested host name
7. Display the IP address.
8. Stop.

PROGRAM
DNSServer
```
Java import java.io.*;
import java.net.*;
Public class udpdns server
{
Private static int index Of(String[]array,String str)
{
str =str.trim();
for(inti=0;i<array.length;i++)
{
if(array[i].equals(str))
return i;
}
return-1;
}
Public static void main(String arg[])throws IO Exception
{
String[] hosts = {"yahoo.com", "gmail.com","cricinfo.com", "facebook.com"};
String[]ip={"68.180.206.184","209.85.148.19","80.168.92.140","69.63.189.16"};
System.out.println("Press Ctrl +C to Quit");
```

```
while(true)
{
Datagram Socket server socket=new Datagram Socket(1362);
byte[] send data = new byte[1021];
byte[]receive data=new byte[1021];
Datagram Packetrecvpack=new Datagram Packet(receive data,receive data.length); server
socket.receive(recv pack);
String  sen = new String(recv pack.get Data());
Inet Address ip address=recvpack.getAddress();
int port = recvpack.getPort();
String cap sent;
System.out.println("Request for host"+sen);
if(index Of (hosts, sen) != -1)
Cap sent=ip[index Of(hosts,sen)];
else
Cap sent = "Host Not Found";
send data=cap sent.get Bytes();
Datagram Packetpack=new DatagramPacket(send data,send data.length,ip address,port);
 server socket.send(pack);
server socket.close();
}}}
```

UDP DNS Client

```
java import java.io.*;
import java.net.*;
Public class udp dns client
{
Public static void main(String args[])throws IOException
{
Buffered Reader br=new Buffered Reader(new Input Stream Reader(System.in));
Datagram Socket client socket = new Datagram Socket();
Inet Address ipaddress;
if (args.length == 0)
Ip address=Inet Address.get LocalHost();
else
Ip address=InetAddress.get By Name(args[0]);
byte[] send data = new byte[1024];
byte[]receive data=new byte[1024];
int portaddr = 1362;
System.out.print("Enterthehostname:");
String sentence = br.readLine();
Send data=sentence.get Bytes();
Datagram Packet pack=new Datagram Packet(send data,send data.length, ip
address,port addr);
Client socket.send(pack);
Datagram Packet recvpack=newDatagram Packet(receive data,receive data.length); client
socket.receive(recvpack);
String modified=new String(recvpack.getData());
```

```
System.out.println("IP Address: " + modified);
client socket.close();
}}
```

OUTPUT

Server

```
Javac udpdnsserver.java
java udpdnsserver
Press Ctrl+C to Quit Request for host yahoo.com
Request for host cricinfo.com
Request for host youtube.com
```

Client

```
>javac udpdns client.java
>java udpdns client
Enter the host name:yahoo.com
IP Address: 68.180.206.184
>java udpdnsclient
Enter the host name:cricinfo.com
IP Address: 80.168.92.140
>java udpdnsclient
Enter the host name:youtube.com
IP Address: Host Not Found
```

RESULT:

Thus the java program for Simulation of DNS using UDP Sockets have written and executed successfully.

Ex.No:5	Using a tool like Wireshark to capture packets and examine the packets
Date:	

AIM: To View and capture network using Wireshark.

PROCEDURE

What is Wireshark?

Wireshark is a sniffer,as well as a packet analyzer.

What does that mean?

- You can think of a **sniffer** as a measuring device. We use it to examine what's going on inside a network cable, or in the air if we are dealing with a wireless network. A sniffer shows us the data that passes through our network card. But Wireshark does more than that.

- A sniffer could just display a stream of bits - ones and zeroes, that the network card sees. Wireshark is also a**packer analyzer**that displays lots of meaningful data about the frames that it sees.Wireshark is an open-source and freetool,and is widely used to analyze network traffic.

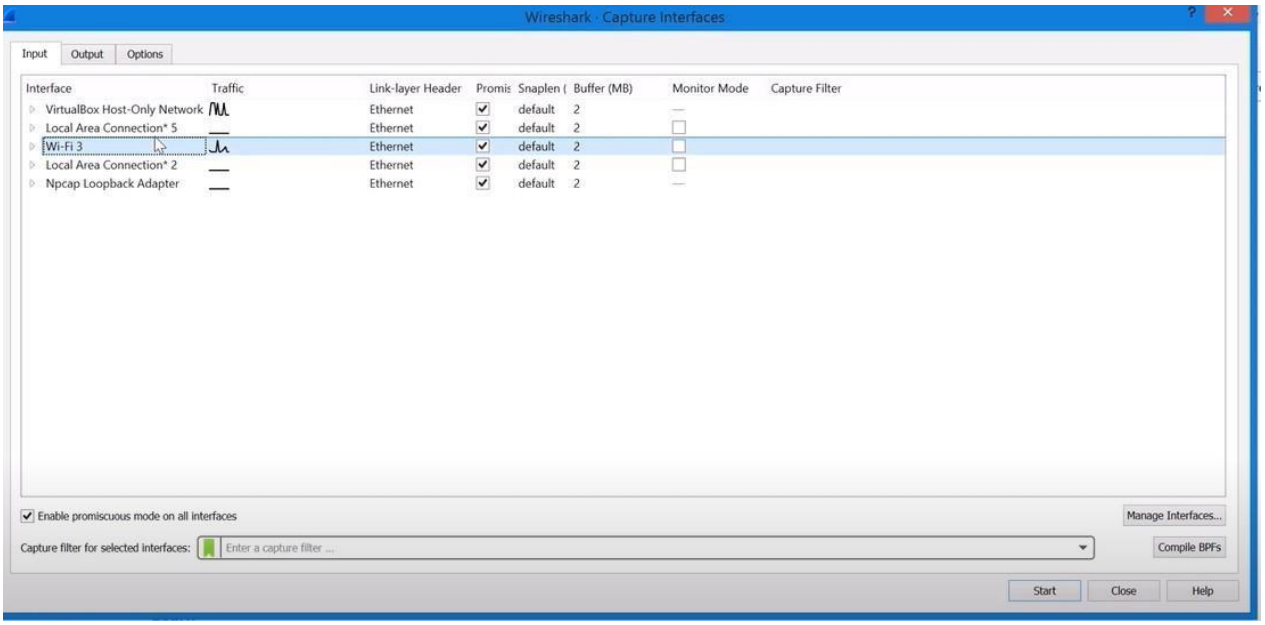
- Wireshark can be helpful in many cases. It might be helpful for debugging problems in your network, for instance – if you can't connect from one computer to another, and want to understand what's going on. It can also help programmers. For example, imagine that you were implementing a chat program between two clients, and something was not working. In order to understand what exactly is being sent, you may use Wireshark to see the data transmitted over the wire.So,let'sget to know Wireshark.

How toDownload andInstallWireshark

Start by downloading Wireshark from its official website:
https://www.wireshark.org/#download
Follow the instructions on the installer and you should be good to go.

How to Sniff Traffic with Wireshark

Launch Wireshark,and start by sniffing ome data.For that,youcanhit `Ctrl+K`(PC) or`Cmd+K`(Mac)to get the `Capture Options`window. Notice that you can reach this window in other ways.You can goto `Capture->Options`.Alternatively,you can click the `Capture Options`icon. I encourage you to use keyboard shortcuts and get comfortable with them right from the start, as they'll allow you to save time and work more efficiently. So,again,I'veused`Ctrl+K`(or`Cmd+K`)and got this screen:



The Capture Options window inWireshark (Source:Brief)

Here we can see a list of interfaces, and I happen to have quite a few. Which one is relevant? If you're not sure at this point, you can look at the Traffic column, and see which interfaces currently have traffic. Here we can see that Wi-Fi 3has got traffic going through it, as the line is high. Select the relevant network interface, and then hit Enter, or click the button Start. Let Wireshark sniff the network for a bit, and then stop the sniff using Ctrl+E/Cmd+E. Again, this can be achieved in other ways – such as going to Capture->Stop or clicking the Stop icon.

RESULT:

Thus the Wireshark have been used to view and capture network traffic.

Ex No: 6(a)	Address Resolutuion Protocol (ARP) using UDP
Date:	

Aim:
To write a java program for simulating ARP protocols using UDP

ALGORITHM:
Client
1. Start the program
2. Create socket and establish connection with the server.
3. Get the IP address to be converted in to MAC address from the user.
4. Send this IP address to server.
5. Receive the MAC address for the IP address from the server.
6. Display the received MAC address
7. Terminate the connection

Server
1. Start the program
2. Create the socket,bind the socket created with IP address and port number and make it a listening socket.
3. Accept the connection request when it is requested by the client.
4. Server maintains the table in which IP and corresponding MAC addresses are stored.
5. Receive the IP address sent by the client.
6. Retrieve the corresponding MAC address for the IP address and send it to the client.
7. Close the connection with the client and now the server becomes a listening server waiting for the connection request from other clients
8. Stop

PROGRAM
Client:

```
import java.io.*;
import java.net.*;
import java.util.*;
class Clientarp
{
Public static void main(String args[])
{
try
{
Buffered Readerin=newBuffered Reader(newInputStream Reader(System.in));
Socket clsct=new Socket("127.0.0.1",139)
DataInputStream din=new DataInputStream(clsct.getInputStream());
DataOutputStream dout=newDataOutput Stream(clsct.getOutput Stream());
System.out.println("Enter the Logical address(IP):");
Stringstr1=in.readLine();
dout.write  Bytes(str1+'\n';
Stringstr=din.readLine();
System.out.println("ThePhysical Addressis:"+str);
```

```
clsct.close();
  }
  catch(Exceptione)
  {
  System.out.println(e);
  }}
  }
```

Server:

```
import  java.io.*;
importjava.net.*;
importjava.util.*;
class Serverarp
{
Public static void main(String args[])
{
try{
Server Socket obj=new
Server Socket(139);
Socket obj1=obj.accept();
while(true)
{
DataInput Stream din=new DataInput Stream(obj1.getInput Stream());
DataOutput Streamdout=newDataOutput Stream(obj1.getOutputStream());
String str=din.readLine();
String ip[]={"165.165.80.80","165.165.79.1"};
String mac[]={"6A:08:AA:C2","8A:BC:E3:FA"};
for(inti=0;i<ip.length;i++)
{
if(str.equals(ip[i]))
{
dout.write Bytes(mac[i]+'\n');
break;
}
}
obj.close();
}
}
catch(Exceptione)
{
System.out.println(e);
}}
}
```

Output:

```
E:\networks>java  Serverarp
E:\networks>java   Clientarp
EntertheLogical address(IP):
165.165.80.80
The Physical Address is: 6A:08:AA:C
```

Result:

Thus the program for implementing to display simulating ARP protocols using UDP was executed successfully and output is verified.

Ex No: 6(b)	Reverse Address Resolution Protocol(RARP) using UDP
Date:	

Aim:
To write a java program for simulating RARP protocols using UDP

ALGORITHM:
Client
1. Start the program
2. Create datagram socket
3. Get the MAC address to be converted in to IP address from the user.
4. Send this MAC address to server using UDP datagram.
5. Receive the datagram from the server and display the corresponding IP address.
6. Stop

Server
1. Start the program.
2. Server maintains the table in which IP and corresponding MAC addresses are stored.
3. Create the datagram socket
4. Receive the datagram sent by the client and read the MAC address sent.
5. Retrieve the IP address for the received MAC address from the table.
6. Display the corresponding IP address.
7. Stop

PROGRAM:
Client:

```
Import   java.io.*;
import  java.net.*;
import java.util.*;
class Clientrarp12
{
Public static void main(String args[])
{

try
{
Datagram Socket client=new Datagram Socket();
Inet Address addr=InetAddress.getByName("127.0.0.1");
byte[] sendbyte=new byte[1024];
byte[]receivebyte=newbyte[1024];
Buffered Readerin=new Buffered Reader(newInput Stream Reader(System.in));
System.out.println("Enter the Physical address (MAC):")
String str=in.readLine();
Send byte=str.getBytes();
Datagram Packet sender=newDatagram Packet(sendbyte,sendbyte.length,addr,1309);
client.send(sender);
Datagram Packet receiver=newDatagram Packet(receive byte,receive byte.length);
client.receive(receiver);
String s=new String(receiver.getData());
System.out.println("TheLogical Address is(IP):"+s.trim());
client.close();
}
```

```
  catch(Exceptione)
{
  System.out.println(e);
  }}}
```

Server:

```
  import  java.io.*;
  import  java.net.*;
  import java.util.*;
  class Serverrarp12
  {
  Public static void main(String args[])
  {
  try{
  Datagram Socket server=newDatagram Socket(1309);
  while(true)
  {
  byte[] sendbyte=new byte[1024];
  byte[]receivebyte=newbyte[1024];
  Datagram Packet receiver=new Datagram Packet(receive byte,receive byte.length);
  server.receive(receiver);
  String str=new String(receiver.getData());
  String s=str.trim();
  InetAddress addr=receiver.getAddress();
  int port=receiver.getPort();

  String ip[]={"165.165.80.80","165.165.79.1"};
  String mac[]={"6A:08:AA:C2","8A:BC:E3:FA"};
  for(inti=0;i<ip.length;i++)
  {
  if(s.equals(mac[i]))
  {
  Send byte=ip[i].getBytes();
  DatagramPacke tsender=new
  Datagram Packet (sendbyte,sendbyte.length,addr,port);
  server.send(sender);
  break;
  }}
  break;
  }}}catch(Exceptione)
  {
  System.out.println(e);
  }}}
```

Output:

I:\ex>java Serverrarp12
I:\ex>java Clientrarp12
Enter the Physical address(MAC):
6A:08:AA:C2
The Logical Address is (IP):165.165.80

RESULT:

Thus the program for simulating RARP protocols using UDP was executed successfully and output is verified.

Ex No:7	Network simulator (NS) and Simulation of Congestion Control Algorithms using NS
Date:	

AIM:

To Study Network simulator (NS) and Simulation of Congestion Control Algorithms using NS

NETWORK SIMULATOR (NS2)

Ns Overview

- Ns Status
- Periodical release(ns-2.26,Feb2003)
- Platform support
- Free BSD,Linux,Solaris,Windows and Mac

Ns functionalities

Routing,Transportation,Traffic sources,Queuing disciplines,QoS

Congestion Control Algorithms

- Slow start
- Additive increase/multiplicative decrease
- Fast retransmit and Fast recovery

Case Study:A simple Wireless network.

Ad hoc routing, mobile IP, sensor-MAC

Tracing, visualization and various utilitie

NS(Network Simulators)

- Most of the commercial simulators are GUI driven, while some network simulators are CLI driven. The network model / configuration describes the state of the network (nodes,routers, switches, links) and the events (data transmissions, packet error etc.). An important output of simulations are the trace files. Trace files log every packet, every event that occurred in the simulation and are used for analysis.Networksimulator scan also provide other tools to facilitate visual analysis of trends and potential trouble spots.

- Most network simulators use discrete event simulation, in which a list of pending "events" is stored, and those events are processed in order, with some events triggering future Events such as the even to f the arrival of a packet a tone node triggering the even to f the arrival of that packet at a downstream node. Simulation of networks is a very complex task.

- For example, if congestion is high,then estimation of the average occupancy is challenging because of high variance. To estimate the likelihood of a buffer overflow in a network, the time required for an accurate answer can be extremely large. Specialized techniques such as "control variates"and "importance sampling"

Have been developed to speed simulation.

Examples of network simulators

There are many both free/open-source and proprietary network simulators.Examples of notable network simulation software are, ordered after how often they are mentioned in research papers:

1. ns(open source)
2. OPNET(proprietary software)
3. NetSim(proprietary software)

Uses of network simulators

Network simulators serve a variety of needs. Compared to the cost and time involved in setting up an entire test bed containing multiple networked computers, routers and data links, network simulators are relatively fast and inexpensive. They allow engineers, researchers to test scenarios that might be particularly difficult or expensive to emulate using real hardware –for instance,simulating a scenario with sever all nodes or experimenting with an new protocol in the network.

Network simulators are particularly useful in allowing researchers to test new networking protocols or changes to existing protocols in a controlled and reproducible environment. A typical network simulator encompasses a wide range of networking technologies and can help the users to build complex networks from basic building blocks such as a variety of nodes and links. With the help of simulators, one can design hierarchical networks using various types of nodes like computers, hubs, bridges, routers, switches, links, mobile units etc.

Various types of Wide Area Network (WAN) technologies like TCP, ATM, IP etc. and Local Area Network(LAN) technologies like Ethernet,tokenrings etc., can all be simulated with a typical simulator and the user can test, analyze various standard results apart from devising some novel protocol or strategy for routing etc.Network simulators are also widely used to simulate battlefield networks in Network-centric warfare.

There are a wide variety of network simulators, ranging from the very simple to the very complex. Minimally, a network simulator must enable a user to represent a network topology, specifying the nodes on the network, the links between those nodes and the traffic between the nodes. More complicated systems may allow the user to specify everything about the protocols used to handle traffic in a network. Graphical applications allow users to easily visualize the working soft their simulated environment.Text-based applications may provide a less intuitive interface, but may permit more advanced forms of customization.

Packet loss

Packet loss occurs when one or more packets of data travelling across a computer network fail to reach their destination. Packet loss is distinguished as one of the three main error types encountered in digital communications;the other two being bit error and spurious packets caused due to noise. Packets can be lost in a network because they may be dropped when a queue in the network node overflows. The amount of packet loss during the steady state is another important property of a congestion control scheme. The larger the value of packet loss, the more difficult it is for transport layer protocols to maintain high bandwidths, the sensitivity to loss of individual packets, as well as to frequency and patterns of loss among longer packet sequences is strongly dependent on the application itself.

Throughput
Throughput is the main performance measure characteristic, and most widely used. In communication networks,such as Ethernet or packet radio,through put or network throughput is the average rate of successful message delivery over a communication channel. Throughput is usually measured in bits per second (bit/s or bps), and sometimes in data packets per second or data packets per time slot.This measure show so on the receiver is able to get a certain amount of data send by the sender. It is determined as the ratio of the total data received to the end to end delay. Throughput is an important factor which directly impacts the network performance.

Delay
Delay is the time elapsed while a packet travels from one point e.g., source premise or networkingresstodestinationpremiseornetworkdegrees.Thelargerthevalueofdelay,the more difficult it is for transport layer protocols to maintain high band widths. We will calculate End to end delay

Queue Length
A queuing system in networks can be described as packets arriving for service, waiting for service if it is not immediate,and if having waited for service,leaving the system after being served. Thus queue length is very important characteristic to determine that how well the active queue management of the congestion control algorithm has been working.

Congestion control Algorithms
Slow-start is used in conjunction with other algorithms to avoid sending more data than the network is capable of transmitting,that is,to avoid causing network congestion.The additive increase/multiplicative decrease (AIMD) algorithm is a feedback control algorithm. AIMD combines linear growth of the congestion window with an exponential reduction when a congestion takes place. Multiple flows using AIMD congestion control will eventually converge to use equal amounts of a contended link. Fast Retransmit is an enhancement to TCP that reduces the time a sender waits before retransmitting a lost segment.

Program:

```
include<wifi_lte/wifi_lte_rtable.h>
struct r_hist_entry *elm, *elm2;
int num_later=1;
elm=STAILQ_FIRST(&r_hist_);
while(elm!=NULL&&num_later<=num_dup_acks_){
num_later;
elm=STAILQ_NEXT(elm, linfo_);
}
if(elm!=NULL){
elm=findDataPacketInRecvHistory(STAILQ_NEXT(elm,linfo_)); if
(elm != NULL){
elm2=STAILQ_NEXT(elm,linfo_);  while(elm2
!= NULL){
if (elm2->seq_num_ < seq_num && elm2->t_recv_
<time){
STAILQ_REMOVE(&r_hist_,elm2,r_hist_entry,linfo_);
delete elm2;
```

```
} else
elm =elm2;
elm2=STAILQ_NEXT(elm, linfo_);
}
}
}
}
void DCCPTFRCAgent::removeAcksRecvHistory(){
structr_hist_entry*elm1=STAILQ_FIRST(&r_hist_);
struct r_hist_entry *elm2;
int num_later=1;
while(elm1!=NULL&&num_later<=num_dup_acks_){
num_later;
elm1=STAILQ_NEXT(elm1, linfo_);
}
if(elm1==NULL)
return;
elm2=STAILQ_NEXT(elm1,linfo_);
while(elm2 != NULL){
if (elm2->type_ == DCCP_ACK){
STAILQ_REMOVE(&r_hist_,elm2,r_hist_entry,linfo_);
delete elm2;
}else {
elm1= elm2;
}
elm2=STAILQ_NEXT(elm1, linfo_);
}
}
inliner_hist_entry
*DCCPTFRCAgent::findDataPacketInRecvHistory(r_hist_entry*start){
while(start != NULL && start->type_ == DCCP_ACK)
start=STAILQ_NEXT(start,linfo_);
return start;
```

Result:

Thus we have Studied Network simulator (NS)and Simulation of Congestion Control Algorithms using NS.

ExNo:8	TCP/UDP performance using Simulation tool.
Date:	

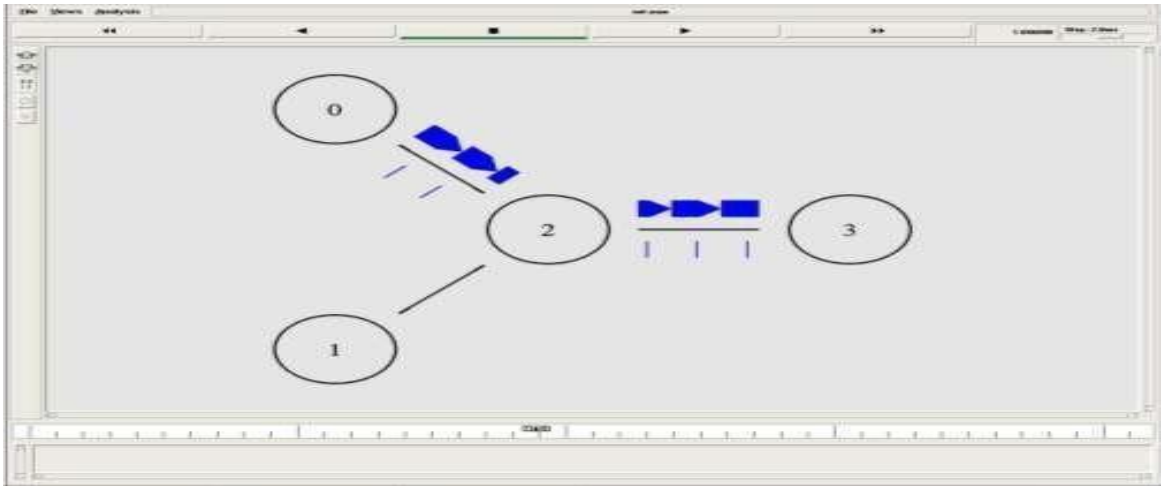
AIM:
To simulate the performance of TCP/UDP using NS2.

TCP Performance
Algorithm
1. Create a Simulator object.
2. Set routing as dynamic.
3. Open the trace and name trace files.
4. Define the finish procedure.
5. Create nodes and the links between them.
6. Create the agents and attach them to the nodes.
7. Create the applications and attach them to the tcp agent.
8. Connect tcp and tcp sink.
9. Run the simulation.

PROGRAM:
```
Set ns[new Simulator]
$ns color 0 Blue
$ns color 1 Red
$nscolor2Yellow
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
Set f [opentcpout.tr w]
$ns trace-all$f
setnf[opentcpout.namw]
$ns name trace-all$nf
$ns duplex-link$n0 $n25Mb 2ms DropTail
$ns duplex-link$n1 $n25Mb 2ms DropTail
$ns duplex-link$n2 $n31.5Mb 10ms DropTail
$nsduplex-link-op$n0$n2orientright-up
$nsduplex-link-op$n1$n2orientright-down
$nsduplex-link-op$n2$n3orientright
$nsduplex-link-op$n2$n3queuePos0.5 set
tcp [new Agent/TCP]
$tcp setclass_ 1
Set sink[newAgent/TCPSink]
$ns attach-agent$n1$tcp
$ns attach-agent$n3 $sink
$ns connect$tcp$sink
Set ftp[newApplication/FTP]
$ftp attach-agent$tcp
$ns at1.2"$ftpstart"
$ns at1.35"$ns detach-agent
$n1$tcp;
```

```
    $ns detach-agent $n3$sink"
$nsat3.0"finish"  proc
finish {} { global ns f
nf
    $nsflush-trace
    close $f
    close$nf
    puts"Running nam.."
    execxgraphtcpout.tr-geometry600x800& exec
    nam tcpout.nam &
    exit0
    }
    $ns run
```

OUTPUT:



UDPPerformance

ALGORITHM :

1. Create a Simulator object.
2. Set routin gas dynamic.
3. Open the trace and namtrace files.
4. Define the finish procedure.
5. Create nodes and the links between them.
6. Create the agents and attach them to the nodes.
7. Create the applications and attach them to the UDP agent.
8. Connect udp and null agents.

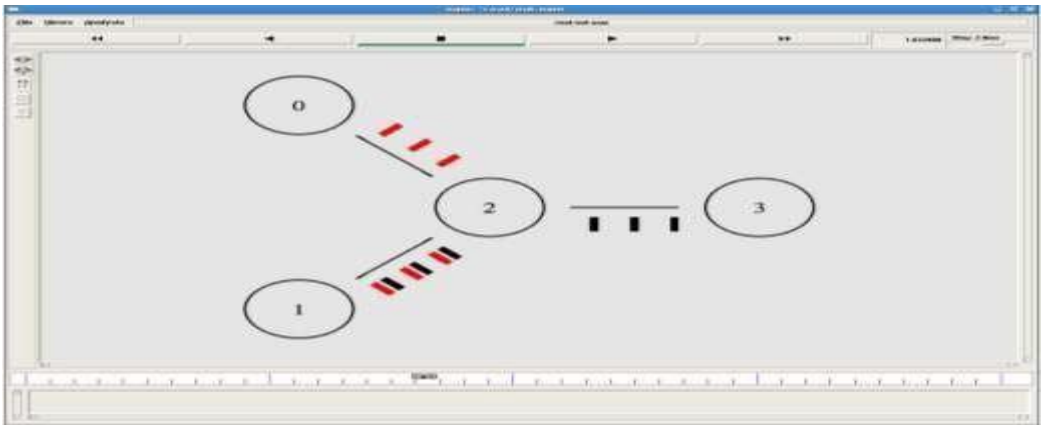
9. Run the simulation.

PROGRAM:

```
Set ns[new Simulator]
$ns color 0 Blue
$ns color 1 Red
$nscolor2Yellow
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
Set f[open udpout.trw]
$ns trace-all$f
Set nf[open udpout.nam w]
$ns namtrace-all$nf
$ns duplex-link$n0 $n25Mb 2ms Drop Tail
$ns duplex-link$n1 $n25Mb 2ms Drop Tail
$ns duplex-link$n2 $n31.5Mb 10ms Drop Tail
$nsduplex-link-op$n0$n2orientright-up
$nsduplex-link-op$n1$n2orientright-down
$nsduplex-link-op$n2$n3orientright
$nsduplex-link-op$n2$n3queuePos0.5 set
udp0 [new Agent/UDP]
$nsattach-agent$n0 $udp0
setcbr0[new Application/Traffic/CBR
$cbr0attach-agent $udp0
setudp1[newAgent/UDP]
$nsattach-agent$n3 $udp1
$udp1setclass_ 0
setcbr1[new Application/Traffic/CBR]
$cbr1 attach-agent $udp1
setnull0[newAgent/Null]
$nsattach-agent$n1$null0
set null1 [new Agent/Null]
$nsattach-agent$n1 $null1
$nsconnect$udp0$null0
$nsconnect$udp1$null1
$nsat1.0"$cbr0 start"
$nsat1.1"$cbr1 start"
puts[$cbr0setpacketSize_] puts
[$cbr0 set interval_]
$nsat3.0"finish"
proc finish {} {
global ns f nf
$nsflush-trace
close $f
close$nf
puts "Running nam.."
execnamudpout.nam&
exit 0
}
```

$ns run

OUTPUT:



RESULT:

Thus the study of TCP/UDP performance is done successfully.

Ex No: 9(a)	Simulation of Distance Vector Routing Algorithm
Date:	

Aim:
To simulate and study the Distance Vector routing algorithm using simulation.

Algorithm:
1. Create a simulator object
2. Define different Colors for different data flows
3. Open a namtrace file and define finish procedure then close the tracefile,and execute nam on trace file.
4. Create n number of nodes using for loop
5. Create duplex links between the nodes
6. Set upUDP Connection between n(0) and n(5)
7. Set up another UDP connection between (1) and n(5)
8. Apply CBR Traffic over both UDP connections
9. Choose distance vector routing protocol to transmit data from sender to receiver.
10. Schedule events and run the program.

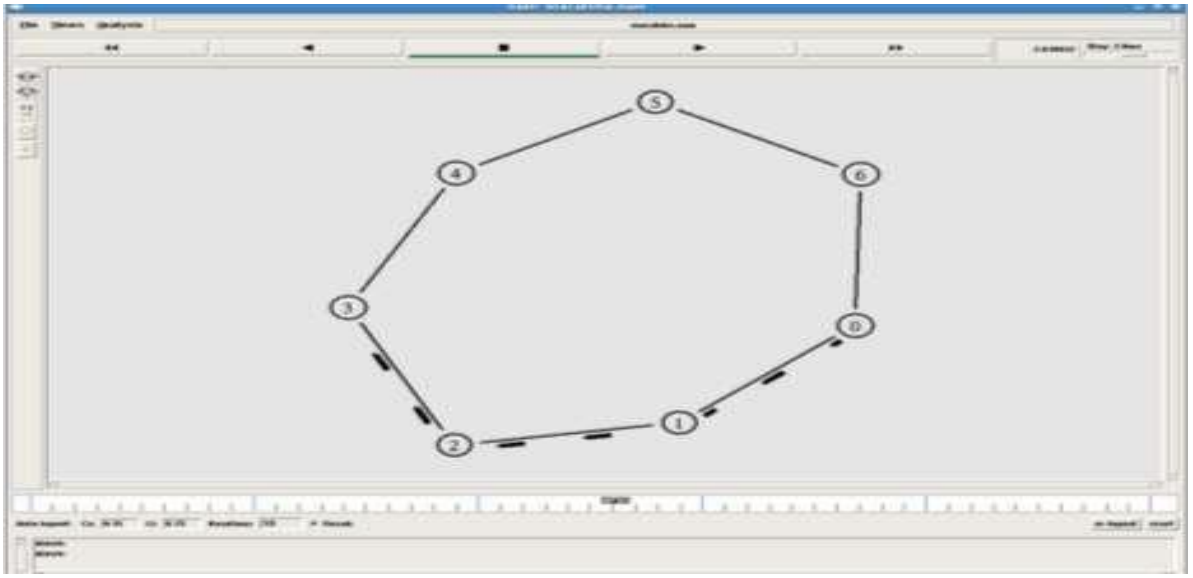
Program:

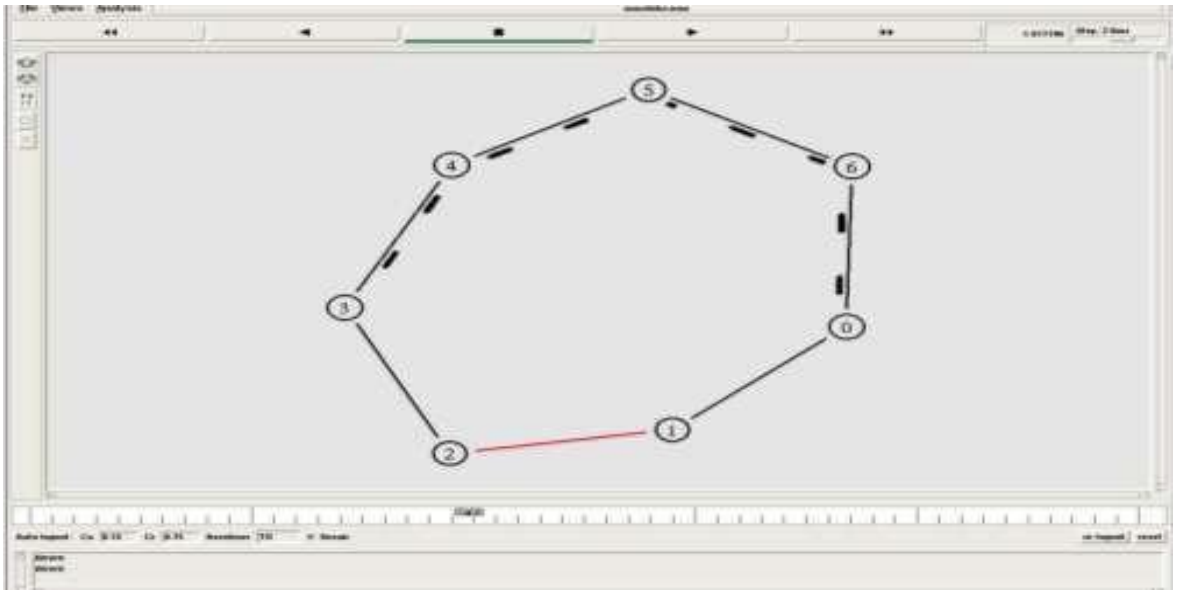
```
Set ns[new Simulator]
set nr [open thro.tr w]
$ns trace-all$nr
Set nf[openthro.nam w]
$ns namtrace-all$nf proc finish{}{
global ns nr nf
$ns flush-trace close$nf
close $nr
Exec namthro.nam&
exit 0
}
for{seti0}{$i<12} {incri1}{ set n($i)
[$ns node]}
for{set i 0} {$i <8} {incri} {

$ns duplex-link$n($i)$n([expr$i+1])1Mb10msDropTail }
$ns duplex-link$n(0)$n(8)1Mb10ms DropTail
$ns duplex-link$n(1)$n(10)1Mb10ms DropTail
$ns duplex-link$n(0)$n(9)1Mb10ms DropTail
$ns duplex-link$n(9)$n(11)1Mb10ms DropTail
$ns duplex-link$n(10)$n(11)1Mb10ms DropTail
$ns duplex-link$n(11)$n(5)1Mb10msDropTail set
udp0 [new Agent/UDP]
$ns attach-agent$n(0) $udp0
Set cbr0[new Application/Traffic/CBR]
$cbr0setpacketSize_500
$cbr0setinterval_0.005
$cbr0attach-agent
```

```
    $udp0setnull0[new
    Agent/Null]
    $ns attach-agent$n(5)$null0
    $ns connect $udp0 $null0
    set udp1[newAgent/UDP]
    $ns attach-agent$n(1) $udp1
    setcbr1[newApplication/Traffic/CBR]
    $cbr1setpacketSize_500
    $cbr1setinterval_0.005
    $cbr1attach-agent
    $udp1setnull0[new
    Agent/Null]
    $ns attach-agent$n(5)$null0
    $ns connect$udp1$null0
    $ns rtproto DV
    $ns rtmodel-at10.0down$n(11) $n(5)
    $ns rtmodel-at15.0 down$n(7)$n(6)
    $ns rtmodel-at30.0up$n(11)$n(5)
    $ns rtmodel-at20.0up$n(7)$n(6)
    $udp0set fid_ 1
    $udp1set fid_ 2
    $ns color 1 Red
    $ns color2 Green
    $nsat1.0"$cbr0 start"
    $nsat2.0"$cbr1 start"
    $nsat 45 "finish"
$ns run
```

OUTPUT:





RESULT:

Thus the simulation of link distance routing algorithm is successfully executed.

Ex No: 9(b)	Simulation Of Link State Routing Algorithm
Date:	

Aim:
To simulate and study the link state routing algorithm using simulation.

ALGORITHM:
1. Create a simulator object
2. Define different colors for different data flows
3. Open a namtrace file and define finish procedure then close the trace file,and execute nam on trace file.
4. Create n number of nodes using for loop
5. Create duplex links between the nodes
6. Set up UDP Connection between n(0) and n(5)
7. Set up another UDP connection between n(1) and n(5)
8. Apply CBR Traffic over both UDP connections
9. Choose Link state routing protocol to transmit data from sender to receiver.
10. Schedule events and run the program.

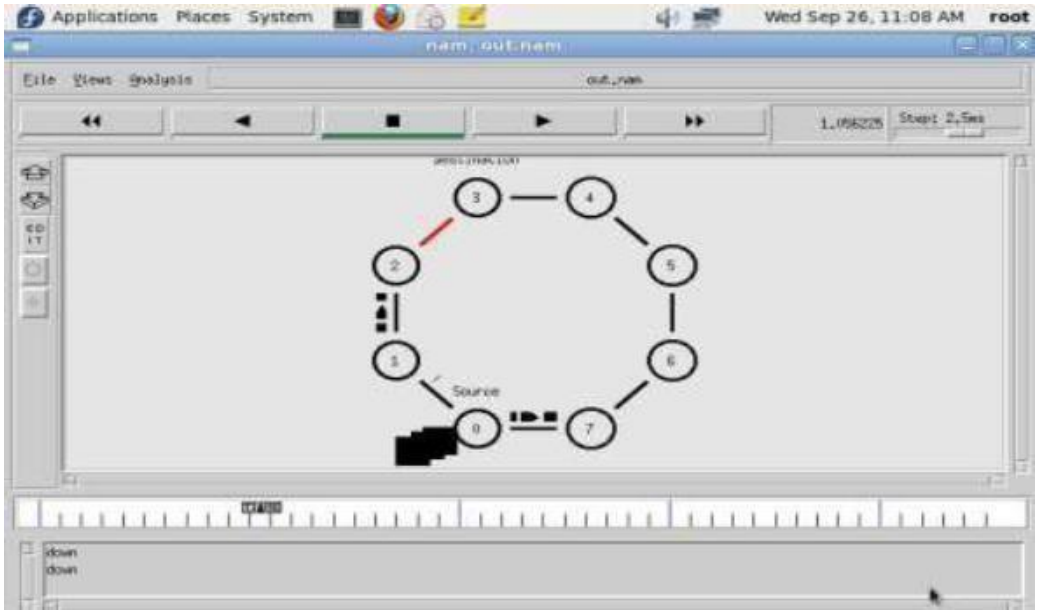
Program:

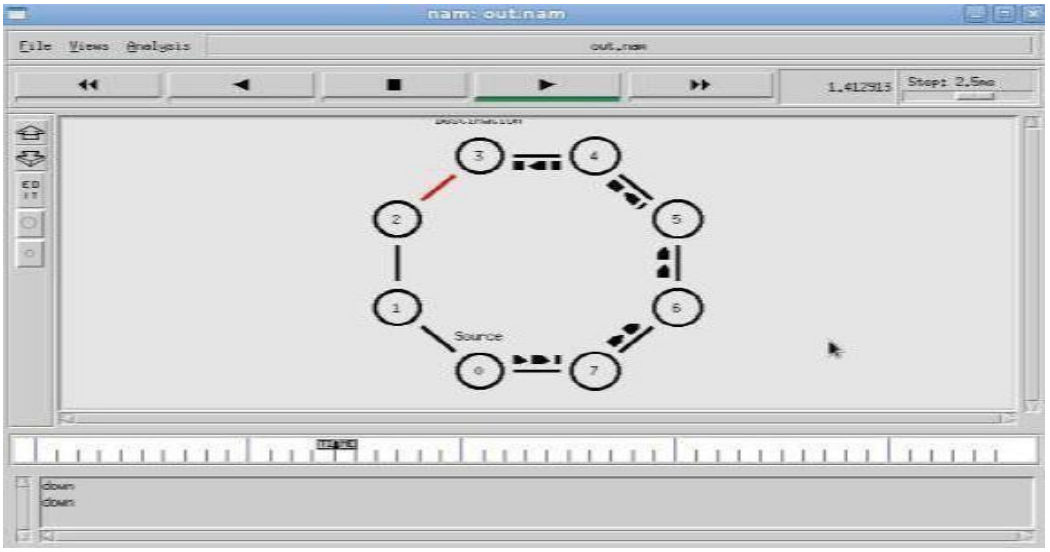
```
Set ns[newSimulator]
set nr [open thro.tr w]
$nstrace-all$nr
Set nf[openthro.nam w]
$ns namtrace-all$nfprocfinish { }{globalnsnrnf
$ns flush-traceclose$nf
close $nr
execnamthro.nam&
exit 0
}
for{seti0}{$i<12} {incri1}{ set n($i)
[$ns node]}
for{set i 0} {$i <8} {incri} {
$ns duplex-link$n($i)$n([expr$i+1])1Mb10msDropTail }
$ns duplex-link$n(0)$n(8)1Mb10ms DropTail
$ns duplex-link$n(1)$n(10)1Mb10ms DropTail
$ns duplex-link$n(0)$n(9)1Mb10ms DropTail
$ns duplex-link$n(9)$n(11)1Mb10ms DropTail
$ns duplex-link$n(10)$n(11)1Mb10ms DropTail
$ns duplex-link$n(11)$n(5)1Mb10msDropTail set
udp0 [new Agent/UDP]
$ns attach-agent$n(0) $udp0
Set cbr0[newApplication/Traffic/CBR]
$cbr0setpacketSize_500
$cbr0setinterval_0.005
$cbr0attach-agent
$udp0setnull0[new
Agent/Null$ns
attach-agent $n(5)
```

```
$null0
$ns connect $udp0 $null0
setudp1[newAgent/UDP]
$nsattach-agent$n(1) $udp1
setcbr1[newApplication/Traffic/CBR]
$cbr1setpacketSize_500
$cbr1setinterval_0.005
$cbr1attach-agent
$udp1setnull0[new
Agent/Null]
$nsattach-agent$n(5)$null0
$nsconnect$udp1$null0
$nsrtprotoLS
$nsrtmodel-at10.0down$n(11) $n(5)
$nsrtmodel-at15.0down$n(7)$n(6)
$nsrtmodel-at30.0up$n(11)$n(5)
$nsrtmodel-at20.0up$n(7)$n(6)
$udp0set fid_ 1
$udp1set fid_ 2
$nscolor 1 Red
$nscolor2 Green
$nsat1.0"$cbr0 start"
$nsat2.0"$cbr1 start"
$nsat 45 "finish"
$ns run
```

OUTPUT:

$ ns distvect.tcl





RESULT:

Thus the simulation of link state routing algorithm is successfully executed.

ExNo:10	Simulation of Error Detection Code (like CRC)
Date:	

AIM:

To implement Error Checking Code using java.

ALGORITHM:

1. Start the Program
2. Given a bit string, append 0S to the end of it (then umber of 0s is the same as the degree of the generator polynomial) let B(x) be the polynomial corresponding to B.
3. Divide B(x)by some agreed on polynomial G(x) (generator polynomial) and determine the remainder R(x). This division is to be done using Modulo 2 Division.
4. DefineT(x)=B(x)–R(x)
5. (T(x)/G(x)=>remainder0)
6. Transmit T,the bit string corresponding toT(x).
7. LetT'represent the bit stream the receiver gets andT'(x)the associated polynomial.The receiver divides T1(x) by G(x). If there is a 0 remainder, the receiver concludes T = T' And no error occurred other wise,the receiver concludes an erroro ccurred and requires a retransmission
8. Stop the Program

PROGRAM:

```
Import java.io.*;
class crc_gen
{
Public static void main(String args[])throws IOException
{
Buffered Readerbr=newBuffered Reader(newInput Stream Reader(System.in));
int[] data;
int[] div;
int[]divisor;
int[] rem;
int[] crc;
intdata_bits,divisor_bits,tot_length;
System.out.println("Enter number of data bits:");
data_bits=Integer.parseInt(br.readLine());
data=new int[data_bits];
System.out.println("Enter data bits:");
for(int i=0; i<data_bits; i++)
data[i]=Integer.parseInt(br.readLine());
System.out.println("Enter number of bits in divisor : ");
divisor_bits=Integer.parseInt(br.readLine());divisor=newint[divisor_bits];
System.out.println("Enter Divisor bits : ");
for(int i=0; i<divisor_bits; i++)
divisor[i]=Integer.parseInt(br.readLine());
System.out.print("Data bits are : ");
for(int i=0; i< data_bits; i++)
System.out.print(data[i]);
System.out.println();
```

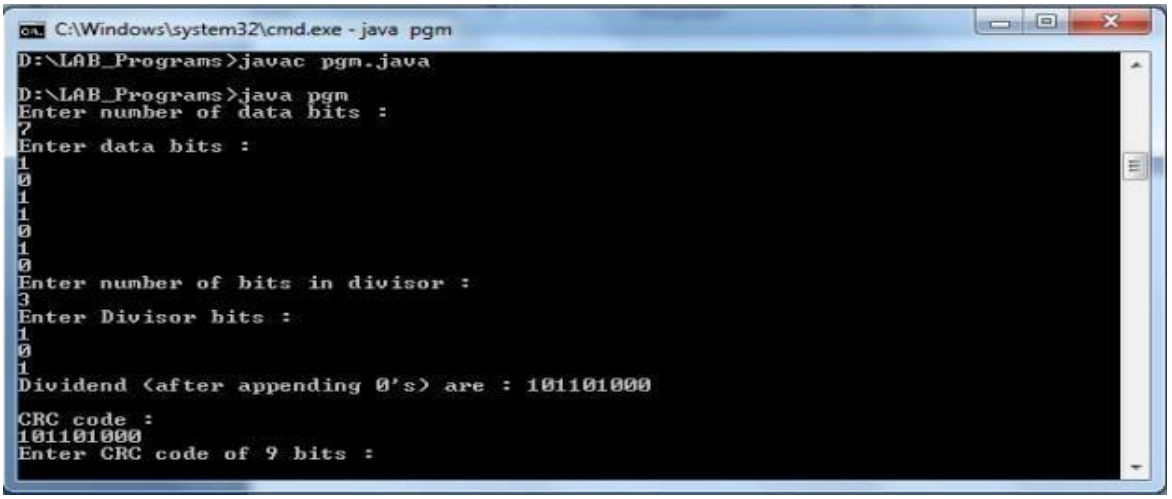
```
System.out.print("divisor bits are : ");

for(inti=0;i<divisor_bits;i++)
System.out.print(divisor[i]);
System.out.println();
*/
tot_length=data_bits+divisor_bits-1;
div=new int[tot_length];
rem=newint[tot_length];
crc=new int[tot_length];
/*.....CRC GENERATION.....*/
for(int i=0;i<data.length;i++)
div[i]=data[i];
System.out.print("Dividend(after appending 0's)are:");for(inti=0;i<div.length;i++)
System.out.print(div[i]);
System.out.println();
for(intj=0;j<div.length;j++){
rem[j] = div[j];
}
rem=divide(div,divisor,rem);
for(int i=0;i<div.length;i++)
{
//appenddividendandremainder
crc[i]=(div[i]^rem[i]);
}
System.out.println();
System.out.println("CRCcode:");
for(int i=0;i<crc.length;i++)
System.out.print(crc[i]);
/*.....ERROR DETECTION.....*/
System.out.println();
System.out.println("Enter CRC code of"+tot_length+"bits:");
for(inti=0;i<crc.length;i++) crc[i]=Integer.parseInt(br.readLine());
System.out.print("crc bits are:");
for(int i=0; i< crc.length; i++)
System.out.print(crc[i]);
System.out.println();
for(intj=0;j<crc.length;j++){
rem[j] = crc[j];
}
rem=divide(crc, divisor, rem);
for(inti=0;i<rem.length;i++)
{
if(rem[i]!=0)
{
System.out.println("Error");
break;
}
if(i==rem.length-1)
System.out.println("No Error");
}
```

```
System.out.println("THANK YOU.....)");

  }
  staticint[]divide(int div[],intdivisor[],intrem[])
  {
  int cur=0;
  while(true)
  {
  for(int i=0;i<divisor.length;i++)
  rem[cur+i]=(rem[cur+i]^divisor[i]);
  while(rem[cur]==0
  &&cur!=rem.length-1)cur++;
  if((rem.length-cur)<divisor.length)break;
  }
  returnrem;
  }
  }
```

OUTPUT:



RESULT:

Thus the above program for error checking code using wa sexecuted successfully.